121 GROUP · CALCULATIONS REFERENCE

How everything is calculated

How every metric, status, and rollup in the app is calculated. Written for the portal dev (so the same numbers land in both builds), the team (so we all agree on what "On Fire" means), and leadership (so the numbers in the report match what's on screen).

Each entry: plain-English summary, formal rule, where it surfaces in the UI, and the source file in code.

A reference for the dev (so the portal calculates the same numbers our build does), team members (so we all agree on what "On Fire" means), and leadership (so the numbers in the report match what's on screen).

Every metric, status, and rollup the app surfaces is listed here. For each one: a plain-English explanation, the formal rule, where it shows up in the UI, and the source file in code.

TL;DR

Two flavors of values in the app:

- **Stored values.** The AM (or another operator) picks them. The system never overrides what a person chose. Examples: each client's status (`fire` / `risk` / `track` / `onboard` / `paused`), each service's progress percentage, each task's severity.
- **Derived values.** Computed live from the underlying data. The system recalculates on every render — no cached or denormalized copies sit on records. Examples: each service's health, every count in NEEDS ATTENTION, the Portfolio Health rollup, day-card load.

When the portal disagrees with our build's numbers, the cause is almost always one of two things: either the portal is using a different *derivation rule*, or it's storing a value the reference treats as derived (or vice versa).

---

1. Status vocabulary

The app uses one canonical set of status names everywhere. Five client-level states, three service-level states.

Client statuses

Status	Meaning	What an AM does with it
`fire` ("On Fire")	The client needs hands-on attention right now. Open tasks are overdue, blocked, or critical.	Open the client, triage the loudest task, get the blocker moving.
`risk` ("At Risk")	Work is drifting. Not on fire yet, but reviews are piling up or schedule is slipping.	Check in this week. Likely a half-hour conversation.
`onboard` ("Onboarding")	Brand-new client still inside the first-30-days setup window.	Run the onboarding checklist. Don't expect production output yet.
`track` ("On Track")	Quietly working. No urgent signal.	Leave alone. The system surfaces them only when something changes.

`paused`	Client is on hold (renewal in flight, payment dispute, vacation freeze). Doesn't count toward "active clients."	Resume when the blocker clears.

`archived` is a separate axis — an archived client is filtered out of every list by default and only resurfaces when the user explicitly opens the Archived filter chip.

Storage model: each client carries a `status` field. The AM sets it manually via the Add Client modal and via the kebab menu on the client detail header. The system never auto-changes it. Type definition: `src/types/flizow.ts:22` (`ClientStatus`).

Service statuses

Same three loudest names as clients (`fire` / `risk` / `track`), but service status is **derived** from tasks. There's no `paused` or `onboard` at the service level.

Status	Rule (in order)
`fire` ("On Fire")	Any open task is overdue (dueDate < today), marked critical severity, or sitting in the blocked column.
`risk` ("At Risk")	Any open task is marked warning severity (drifting work, reviews piling up). No fire signals present.
`track` ("On Track")	Everything else. Includes "no tasks yet" — a brand-new service shouldn't read as a problem.

"Open" excludes archived tasks and tasks already in the `done` column. The first fire signal short-circuits the loop (no need to keep checking other tasks once a fire flag is set).

Source: `src/utils/clientDerived.ts:132` (`serviceHealth`).

---

2. Per-client values

Portfolio Health (Home page)

The three big metric cards at the top of Home — ON FIRE / AT RISK / ON TRACK.

Plain English: count of clients currently in each state. Paused clients aren't shown (they're not in active rotation).

Formal rule:

```
fire = count of clients where status === 'fire'
risk = count of clients where status === 'risk'
track = count of clients where status === 'track'
active = total clients minus paused
```

The "Across N active clients" line above the cards uses `active`. The clicks on each card route to `/clients?view=<state>` and the Clients filter chip with the same state shows the matching count — they share one source of truth.

Source: `src/pages/OverviewPage.tsx:166` (`health` memo).

Subtitle ("X clients need you now" / "Steady morning ahead.")

Adaptive copy below the greeting:

- If `fire > 0`: "X clients need you now."
- Else if there's overdue work: "X tasks overdue across the workspace."
- Else: a time-of-day greeting fallback ("Steady morning ahead.", etc.)

The branch table lives in `pickTagline` (same file).

Client list row — right-hand metric

Each row carries a short status-aware sentence on the far right. The sentence shape depends on the client's status:

Status	Metric text
`fire`	`{n} overdue` (e.g. "5 overdue"). Falls back to "Needs attention" if there's no overdue task yet.
`risk`	`{n} at risk` — count of tasks with severity `warning` or `critical`. Falls back to "At risk".
`onboard`	`{N}% setup` — average progress across the client's services.
`paused`	"Paused"
`track` (or anything else)	"On track"

Source: `src/utils/clientDerived.ts:79` (`clientMetric`).

Last-activity timestamp

The "6d ago" / "Mon" / "Apr 7" stamp on the right of each list row.

Plain English: the most recent activity touching this client. Today, in the demo data, we use the latest `createdAt` of any task tied to the client's services. Falls back to the client's `startedAt` so a brand-new client with no tasks yet still has a date.

Production note: a real system would track this as a denormalized `updatedAt` on the client record, updated whenever any of its services, tasks, notes, or contacts change. The portal should add that field and write it on every relevant mutation — the demo's "latest createdAt" approach is good enough at small scale, but it gets slow when you have thousands of clients.

Source: `src/utils/clientDerived.ts:188` (`clientLastTouched`).

"Xd ago" formatting

The same formatter renders the timestamp in three buckets:

Age	Format
Under 1 minute	"just now"
Under 1 hour	"Xm ago" (minutes)
Under 1 day	"Xh ago" (hours)
Under 7 days	"Xd ago" (days)
Under 30 days	weekday name ("Mon", "Tue") — friendlier than "14d ago"
30+ days	short date ("Apr 7")

Anchored against the store's `today` value (not `Date.now()`) so the output stays stable across re-renders.

Source: `src/utils/clientDerived.ts:205` (`relativeTimeAgo`).

---

3. Per-service values

Service type

Two types: `project` (one-off deliverable that ships once) and `retainer` (recurring monthly scope). Stored on the service record; the AM picks it in the Add Service modal.

Source: `src/types/flizow.ts:35` (`ServiceType`).

Service health

Derived from tasks (rule listed in §1 above). Surfaces as:

- A colored dot or pill on each service row in ACTIVE SERVICES
- The colored health bar on the pinned card on Home (red / amber / green)

Service progress

Plain English: 0–100 number that drives the progress bar on the service card.

Storage model: stored on the service record as `progress`. **Not** derived from task completion. Whoever owns the service updates it manually as work moves.

Source: `src/types/flizow.ts:394` (`Service.progress`).

For the portal: if you want progress to auto-update from task state, that's a feature change that needs leadership sign-off. Our build deliberately keeps it manual because progress on a creative project isn't

linear with task count (the last 20% takes 50% of the time, etc.). The AM judging completion is more accurate than dividing done-tasks by total-tasks.

Service "Next deliverable" date

Stored on the service record as `nextDeliverableAt`. The AM sets it in the Add Service modal and updates it as milestones move. Drives the "Due May 25" subtitle line on each service row in ACTIVE SERVICES (see §4).

Source: `src/types/flizow.ts:396`.

ACTIVE SERVICES eyebrow ("7 of 7 · 2 projects, 5 retainers")

The count + split line under ACTIVE SERVICES on the client detail.

```
total = services.length
projects = services.filter(s => s.type === 'project').length
retainers = services.filter(s => s.type === 'retainer').length
eyebrow = "{total} of {total} · {projects} projects, {retainers} retainers"
```

Note on the "X of X" prefix: right now both numbers are the services-array length. The prefix exists because the original spec wanted "active of total" (excluding archived/paused services) but archive at the service level isn't surfaced in this UI yet, so they're equal. The portal should still render both numbers for shape parity — when archived services land, this slot will start showing the real active count.

Source: `src/pages/ClientDetailPage.tsx:904`.

Service row subtitle ("Template: Brand Refresh · Due May 25")

Two parts separated by a middle dot:

- **Template:** the service's `templateKey` resolved to its display name. If no template assigned: "Template: None".
- **Due:** `nextDeliverableAt` formatted as a short date.

For retainers, the subtitle still shows the template + next deliverable date — the "next deliverable" on a retainer is the next monthly checkpoint.

Service progress label ("Progress 35%" vs "This month 33%")

Same numeric value, different label depending on service type:

- `project` → label reads "Progress" (cumulative completion)
- `retainer` → label reads "This month" (current-month consumption)

The label switch is purely cosmetic — both pull from `service.progress`. The framing differs because the same number *means* something different for the two service types: 35% complete on a one-off deliverable vs. 35% of this month's retainer hours used.

4. Per-task values

Overdue

`task.dueDate < today` AND the task isn't archived AND isn't in the `done` column. Comparison is lexicographic on ISO date strings — works correctly without parsing because the format is sortable.

Source: `src/utils/clientDerived.ts:164` (`countOverdueTasks`).

Blocked

`task.columnId === 'blocked'`. The blocked column is a fixed lane on every kanban board.

Severity (`warning` / `critical`)

A manually-set field on each task. The owner marks it via the card modal. Drives:

- Service-health derivation (any `critical` → service goes On Fire; any `warning` → service goes At Risk)
- Client list row metric ("N at risk" counts both `warning` and `critical`)

The system never auto-sets severity — it's the operator's read on the work.

---

5. MY TASKS card aggregation (Home)

This section renders one card per fire-or-risk client (not one card per task). Cards are sorted by severity, then by total urgent task count, then by oldest overdue date.

For each card in the feed:

Field	Rule
Severity pill	`client.status` rendered as "On Fire" or "At Risk".
Client name	`client.name`.
Aggregate title	`{N overdue} overdue · {M blocked} blocked` if both > 0. Just the non-zero half if only one. Falls back to "Marked on fire — no blocker logged yet" for fire clients with zero urgent tasks.
Blocking-reason caption	The longest blocker reason on any task tied to the client. Falls back to the primary task title when no blocker reason exists.
Days-overdue stamp (right)	Anchored on the oldest overdue task: `floor((today – oldestDueDate) / 86,400,000)` days. "1 day overdue" / "N days overdue". Falls back to "Blocked" / "Needs review" when nothing is measurably late.

Review button click	Deep-links to the kanban board of the primary task (worst-case first: oldest overdue → first blocked → client detail).
Delegate button click	Opens the Delegate popover anchored to that card.

Empty state: when `health.fire === 0` and no risk clients have unresolved tasks, the section renders "Nothing urgent right now. Enjoy the quiet."

Source: `src/pages/OverviewPage.tsx:1137` (`buildAttentionCards`).

6. NEEDS ATTENTION (Client detail Overview tab)

Two-card block at the top of the Overview tab, computed from the client's open tasks. Each card renders only when its count > 0.

Card	Rule
Past-due card (red flame icon)	Count of open tasks where `severity === 'critical'` OR `columnId === 'blocked'`. Subtitle: name of the first such service + "tap to open". Clicks straight through to that service's board.
At-risk card (gray chat icon)	Count of open tasks where `severity === 'warning'`. Subtitle: name of the first such service + "review soon". Clicks straight through to that service's board.

Timestamp ("As of this morning") signals these are computed at start of day, not refreshed live.

Source: `src/pages/ClientDetailPage.tsx:783` (`buildAttentionChips`).

7. Onboarding rollup (Client detail Onboarding tab)

The progress line on the right of "SETUP & ONBOARDING".

Plain English: how far along are this client's services in their template-driven onboarding checklists.

Formal rule:

```
totalItems = count of onboarding items across all the client's services
doneItems = count of items where done === true
openServices = count of services where doneCount < totalItemsForThatService

# Eyebrow string:
if openServices === 0:
"All set · {doneItems} of {totalItems} items complete"
else:
"{openServices} of {servicesCount} services in progress · {doneItems} of
```

```
{totalItems} items complete"
```

Each service onboarding card also carries its own subtitle showing the template name + items left (e.g. "Brand Refresh · 3 items left").

Source: `src/pages/ClientDetailPage.tsx:1283`.

---

8. Capacity / daily slot load

The "0/6" badge on each day card in MY SCHEDULE, and the green / amber / red zone color.

Load

```
load = sum of (task.slots ?? 1) for every "open" task where the
member is the assigneeId AND the task's dueDate matches
```

"Open" excludes archived tasks AND tasks already in the `done` column — finished work doesn't keep eating capacity.

Multi-owner tasks (`assigneeIds[]`) do NOT contribute to anyone's load. Only the primary `assigneeId` absorbs the slots. If two designers share a job, the AM splits it into two tasks. This intentional rule keeps "who owns this" unambiguous.

Source: `src/utils/capacity.ts:58` (`loadFor`).

Soft cap and Max cap

Two numbers per (member, date) pair, with a resolution chain:

```
1. Per-day override exists for (member, date) → use it
2. Else member has standing capSoft/capMax → use those
3. Else fall back to DEFAULT_CAP_SOFT (6) and DEFAULT_CAP_MAX (8)
```

Soft cap is the comfortable daily target. **Max cap** is the hard ceiling. The per-day cap-edit popover (the kebab on each day card) sets both numbers for a single day — overriding the standing defaults. A holiday or PTO day is modeled by overriding both to 0.

Source: `src/utils/capacity.ts:88` (`effectiveCapFor`). Defaults at lines 29, 33.

Load zone (green / amber / red)

```
load <= soft → green (under target, quiet state)
soft < load <= max → amber (warning)
load > max → red (over capacity)
```

The zone drives the badge color on the day card and the heatmap shading on the Team Capacity Heatmap.

Source: `src/utils/capacity.ts:108` (`loadZone`).

---

9. Service tag chips on the client list row

The three pills under each client name on the Clients page (e.g. "Brand Refresh Q2", "Marketing Site v3", "Conversion Optin").

```
ordered = resolve client.serviceIds into Service records,
in the order the client lists them
if ordered.length <= 3:
visible = ordered service names
overflow = 0
else:
visible = first 3 service names
overflow = ordered.length - 3 (rendered as "+N more")
```

Newest project work shows first because the demo data unshifts new projects to the front of `client.serviceIds`. Production should do the same.

Source: `src/utils/clientDerived.ts:46` (`servicePills`).

10. Notifications

Generated live by `deriveNotifications(data, memberId)` — no stored event log. Every notification has a stable id derived from its source row so the read/dismissed state in localStorage persists across re-derives.

Categories (in order they appear in the bell):

Category	Rule	Audience
Overdue tasks	One per overdue task owned by the current member.	Self (urgent prefs).
Tasks due today	One per task owned by current member where `dueDate === today`.	Self (urgent prefs).
On Fire clients	One per client where `status === 'fire'`.	All operators (urgent prefs).
Daily digest	One synthesized "X items today" notification per day.	All operators (digest pref).
Time-off pending	One per pending time-off request, capped at 6.	Owner + Admin only.
Time-off decided	Recent decisions (last 14 days), capped at 5.	The requester.

Source: `src/data/deriveNotifications.ts`.

11. External / not-derived-by-us

The Stats tab on the client detail pulls these from third-party integrations, not from our database:

- Ad Spend (Google Ads, Meta Ads, LinkedIn Ads, TikTok Ads…)
- Leads (Hubspot, Salesforce, our own form submissions)
- Blended CPL (computed live: total ad spend / total leads)
- Email Engagement (HubSpot, Mailchimp open rates)
- Channel-level cards (per-integration breakdowns)

The "Live · Synced N min ago" indicator shows the freshness of the latest pull. Our build doesn't store these numbers — every refresh hits the integration's API.

The portal should match this exactly: fetch from each integration, display the results, and cache only as long as the freshness indicator says (don't snapshot stale numbers into the database).

How to extend this doc

When a new metric, status, or rollup lands in our build:

1. Add it to the relevant section (clients / services / tasks / rollups / capacity).
2. Use the same shape: plain-English summary, formal rule, where it surfaces in the UI, source file + function.
3. If it's stored (not derived), note it loudly so the portal knows to add the field.
4. If it's derived, paste the rule literally — exact field names, exact comparisons. Ambiguity here causes the portal numbers to drift.

Calculation drift is the most common cause of QA findings between the two builds. Keeping this doc current means we catch divergence in code review rather than four rounds later.